

TEL418: Networks II Lab – Spring 2025

2nd Project – Speedtest with Unix sockets

Team 11

Elli Tzaferi - A.M.: 2020030043

Dimitris Keramidas - A.M.: 2020030028

30th May 2025

1 Introduction

In this report we present our designed Speedtest application that has been developed using UNIX sockets. For our application we created two C files that create a server and a client. We start the server and using the server IPv4 address we connect the client to our server. The server receives data and every 2 seconds prints the throughput for that time interval. The client prints every 2 seconds the total number of bytes sent up to that point and the number of bytes sent in the last 2 seconds. When we reach the 30 second timestamp the client stops sending data and the server closes the connection after not receiving any new data. To evaluate our application we test 5 distinct scenarios that cover the majority of real-world cases and check both the overall throughput for the 30 second test and we do a timeseries analysis for every 2 seconds. Out of these we notice In addition, we used iperf3 tool to verify the functionality of our app as far as throughput is concerned. Out of that we notices Furthermore using our Wi-Fi doctor system from project 1 we analyzed the pcap file we sniffed using Wireshark during each experiment (each scenario). Our systems estimates the throughput, the data rate , the frame loss, the RSSI and the rate gap for each 2-second window inside the pcap file. Based on this analysis we notice Finally to have a better estimation we updated the formula to calculate the estimated throughput to include the channel utilization as following:

$$\text{Throughput} = \text{Data Rate} * (1 - \text{Frame Loss Rate}) * \text{Channel utilization}$$

2 Design: Speedtest App implementation

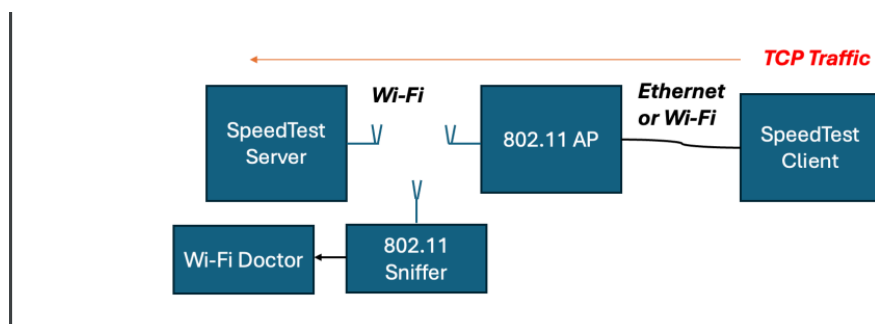
The client side script just sends a bunch of A characters in a while loop with the default buffer size 128kb. On the othe side, the server script waits for one connection and after it accepts recieves and counts every packets length adding them to 2 counters, one for the 2 second throughput and one for the aggregate. The 2 second counter gets reset each interval while the aggregate remains till the connection is closed. Each 2 seconds the throughput gets calculated by the below formula:

$$\text{Throughput} = \frac{\text{interval_bytes} * 8}{\text{time} * 1048576.0}$$

Finally it exits the recieve loop and waits for a new connection. Logs for this script are written in a file in json format.

3 Evaluation: Speedtest App evaluation

The basic topology of the Speedtest App Evaluation was:



To evaluate the functionality of our Speetest App we tested it under 5 scenarios, compared it to a popular market speedtest tool (iperf) and also checked the theoretic throughput we got when running the server-client communication data through our Wi-Fi doctor system (developed in the premises of project 1 of this course). The evaluation of the app through these methods and the results yielded can be found in the next subsections of this report.

3.1 Evaluation settings

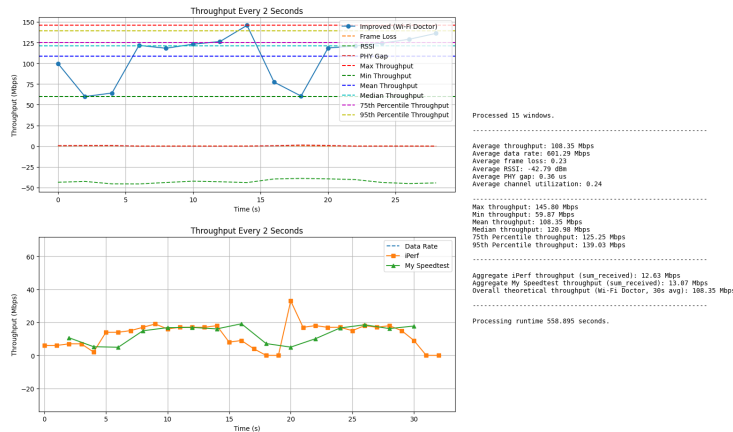
Firstly, we tested 5 scenarios of performing the SpeedTest: (1) Having our server close to our AP and connecting to the 2.4 GHz band of the AP, (2) having our server far from our AP and connecting to the 2.4 GHz band of the AP, (3) having our server close to the AP and connecting to the 5 GHz band of the AP, (4) having our server far from the AP and connecting to the 5 GHz band of the AP and (5) moving at pedestrian speed far from the AP and getting closer at the AP.

In all of the above scenarios, we used 3 laptops. The server was running at the first laptop at a native Linux environment. The packet sniffer (Wireshark) was running at the second laptop, whose Wi-Fi card was set to monitor mode using command line prompts, in a native Linux environment (this Laptop since it had its Wi-Fi card set to monitor mode was offline). Lastly, the client was running at the third laptop in a virtual Linux environment.

Note: For the two 2.4 GHz scenarios, the AP (router) provider settings did not allow us to switch from channel 56 (5 GHz band) to a channel in the 2.4 GHz band. As such we turned one of our mobile phones into an AP (Mobile Hotspot) which was operating in channel 6 (2.4 GHz band). Since there was no Ethernet option for this type of AP, we tried to imitate the Ethernet connection for the client by placing the mobile phone on the laptop running the client. We understand that this could lead to skewed results and we take that into account for the interpretation of the results for these two scenarios.

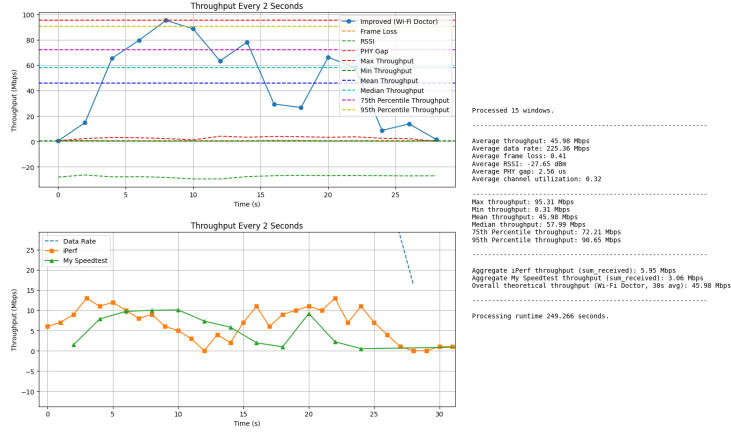
3.1.1 Scenario 1: SpeedTest server device close to the AP at 2.4GHz.

To test this scenario we performed the experiment with our server within less than 0.5 meters from the AP. We run our SpeedTest App for 30 seconds and saved the throughput calculations sent every 2 seconds in a json file. After completing the experiment we went over our data using our Wi-Fi doctor Visualizer and received following plot:



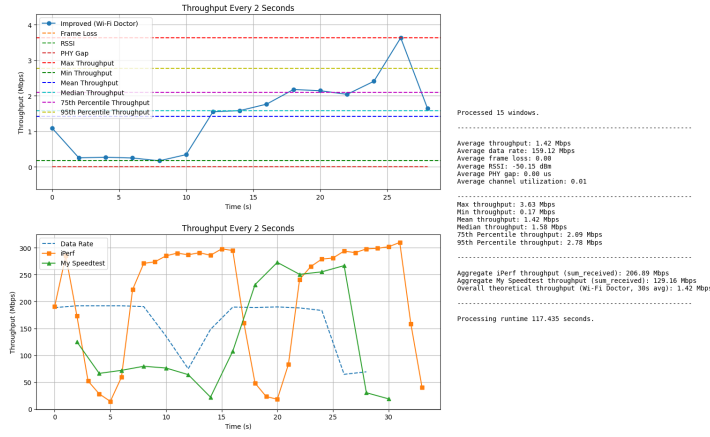
3.1.2 Scenario 2: SpeedTest server device far from the AP at 2.4GHz.

For this scenario we performed the experiment by moving our server as far as possible but still maintaining our connection to the AP (but barely). Again we run the SpeedTest App for 30 seconds and did the data processing using our Wi-Fi doctor system and received following result plot:



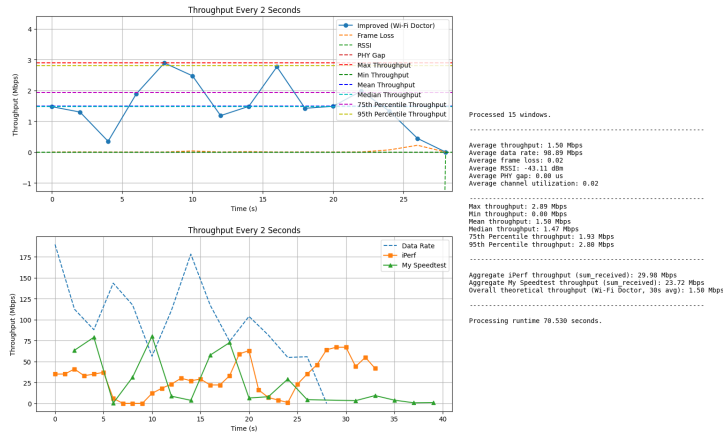
3.1.3 Scenario 3: SpeedTest server device close to the AP at 5GHz.

For this scenario we followed the same procedure and experiment set up as Scenario 1. The yielded result plot is:



3.1.4 Scenario 4: SpeedTest server device far from the AP at 5GHz.

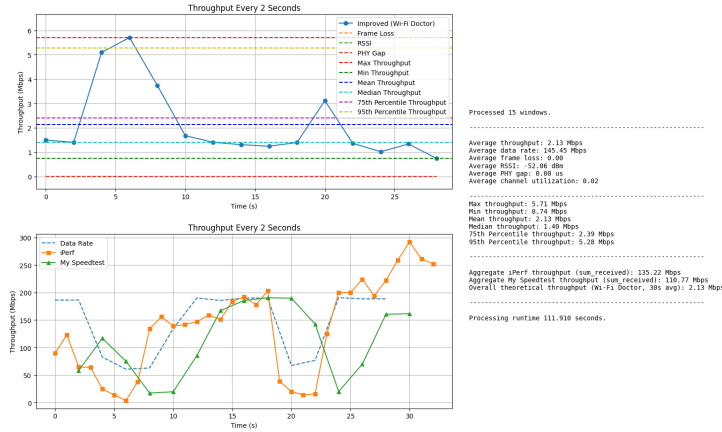
For this scenario we followed the same procedure and experiment set up as Scenario 2. The yielded result plot is:



3.1.5 Scenario 5: Mobility scenario where the SpeedTest server device is moving in pedestrian speed far from the AP.

For this scenario we chose to use our AP working in the 5 GHz band. Due to limitations of the area coverage provided by a home router (our AP) we simulated this experiment by starting with our server being as far as

possible from the AP (like scenarios 2 and 4) and timing our steps to walk towards the AP until we reach as close to the AP as possible. After completing the experiment we again processed our data using our Wi-Fi doctor system and received following result plot:



3.2 Throughput verification: Iperf

To verify that the throughput measured by our SpeedTest App is accurate and close what it should be we used the iperf tool. The iperf tool works on a server-client basis much like our SpeedTest App. After installing the iperf package to both the laptops running the client and the server and completing each one of the scenarios above for our own app we run iperf (Each time we completed a test for our app we run the iperf test under the same conditions). For each iperf test scenario we saved the received throughput data into a json file. After completing all our testings we process each iperf json file along with the corresponding json file of our app results using the Wi-Fi doctor visualizer and plot the results of both tools in the same plot at 2 second intervals. In the plots of the above subsection we can see both results and based on these we have following commentary:

- **General:** In all scenarios we can see our and the iperf SpeedTest while not identical, in our experiments they were run at different times, follow the same bounds and fluctuations validating the correct results of our SpeedTest. Furthermore our implementations graph follows the form of the reported one from the captured pcap, although values vary due to the theoretical nature of our Wi-Fi doctor.
- **Scenario 1:** Due to the fact that 2.4 GHz channel test was conducted with the hot-spot of a mobile phone we can see the theoretical throughput is especially high, that could be a result of the different design of the networks, mobile phone networks are made with the great distances between devices in mind so the would have higher speeds reported than a home network. On the other side due to the larger traffic in those frequencies the actual speeds dropped significantly as we see from the SpeedTests.
- **Scenario 2:** Moving further away from the AP we can see both the theoretical and experimental throughput drop significantly. In our "far from AP" experiment we also had 2 extra walls between the AP and the device witch aligns with the theoretical drop in power of 2.4 GHz networks by 30%.
- **Scenario 3:** On this experiment we can see lower theoretical throughput than expected, that can be due to increased devices in a network leading on the AP choosing suboptimal mcs therefor not utilizing the channel fully and resulting in the huge gap from the experimental throughput.
- **Scenario 4:** As we can expect moving further makes the connection more prone to interference and lowers the signal strength dropping the throughput significantly, while the same problem as in scenario 3 persists.
- **Scenario 5:** Finally while moving we can see the experimental throughput as expected increases. As for the theoretical values they could be skewed due to interference or that the AP didn't gather enough packets to reevaluate the mcs index.

3.3 Wi-Fi Doctor Analysis

To further check our SpeedTest App findings we process the pcap files with the packets sniffed during each scenario-experiment using a modified version of our Wi-Fi doctor system from project 1, to extract a theoretic throughput estimation, the data rate, the frame loss, the RSSI and the phy gap.

3.3.1 Analysis

To do the analysis in a way similar to the way our SpeedTest App works we divide the packets in the pcap files into packets received within a 2-second window according to the timestamp of each packet. Afterwards, we use the fields of the packets in each window to calculate the average of the aforementioned metrics for the window. After processing all the packets in the pcap file (so all the windows) we plot the metrics found for each 2-second window along with an overall throughput average.

Note: The pcap files from Wireshark also hold some packets before and after the 30 second experiment due to human factor in the initiation of the capture and communication that skew the metrics slightly.

3.3.2 Revised Throughput Estimation

As stated in the introduction we decided to improve the throughput estimation of our Wi-Fi doctor system by integrating the channel utilization. As such our new formula is:

$$\text{Throughput} = \text{Data_rate} * (1 - \text{Frame_loss_rate}) * \text{Channel_utilization}$$

We chose to include the channel utilization in our formula so that our estimated throughput can be closer to real-world network conditions. The initial formula (without the channel utilization) assumes that the channel is constantly fully utilized and any packet loss occurs due to frame errors, instead of idle times due to congestion, control overheads or traffic load variability. Adding the channel utilization in the equation, we aim to check the proportion of time the channel is transmitting useful information and take into account the network load conditions and protocol behavior (e.g. collision avoidance algorithms in TCP). Thus, we use the channel utilization to reduce the throughput appropriately according to how effectively the channel is being used.

Note: We notice that the theoretical throughput we receive is largely different than the throughput captured by the SpeedTest App and the iperf tool which can either occur due to the channel utilization factor or the interferences that could be captured in the pcap file as the AP is a home router and there are other devices in the home network that could skew the metrics since we only check packets from the server to the router (AP).

4 References

[IEEE 802.11 Throughput Calculation](#)